Differential Fuzzing Analysis Report by OG Technologies EU

`stellar` (Rust CLI, reference) vs. `stellar-go-cli` (Go implementation)

Stellar Go CLI Security Assessment by Runtime Verification
`docs/audit/differential-fuzzing.md` v0.2

2026-11-12

1 Objective

The goal of this assessment was to check whether the Go implementation handles inputs consistently with the Rust reference CLI. We used differential fuzzing to feed both implementations the same inputs and compare their normalized outputs. We also ran local fuzz tests to check for crashes and violations of expected properties in areas where no reference implementation was available.

The Rust reference environment was:

```
stellar 28.0.0 (300aaf69ab100536678bdb641428b06f06b318ea)
stellar-xdr 28.0.0 (d0f1330e43c3a2c0c616da30698b24140d4eea72)
xdr (9c9c145953e80990d6ff1ae3a6a973a0ce6d0694)
```

The Go environment was `go1.27.2 darwin/arm64`.

For differential tests, the Rust CLI was started as a subprocess for each input. We covered `stellar xdr encode/decode`, `strkey decode`, and `tx hash/decode`. Tests automatically skip the differential checks when the reference binary is not available.

2 Methodology

The test setup follows the CDF approach described by Aumasson (2017). Go's native `testing.F` framework handles input mutation, corpus management and test-case minimization. The Go implementation runs in-process, while the Rust CLI serves as an external reference. Small adapters normalize the outputs so that both sides can be compared.

One practical challenge was that the Go SDK's `xdr.ScVal` does not provide a serde-JSON codec compatible with the Rust CLI. To work around this, we implemented a test-only codec that maps the generated ScVal variants to the tagged JSON format expected by the Rust CLI.

Rather than feeding completely random bytes directly into the XDR decoder, the tests use a structure-aware generator. Fuzz input bytes drive deterministic, depth-bounded generation through `genDec`/`genScVal`. This gives the fuzzer a better chance of exploring meaningful values instead of spending most of its time rejecting invalid length fields.

3 Target Matrix

The following table summarizes the areas covered and the corresponding fuzz targets.

#	Surface	Target(s)	Mode
1	`CompareScVal`/`ToScVal`/ScMap	`FuzzCompareScVal`, `FuzzToScVal`, `FuzzScMapSortDeterminism`	local
2	ScVal JSON↔XDR	`FuzzScValEncodeDifferential`, `FuzzScValDecodeDifferential`	differential
3	Transaction envelope	`FuzzTxEnvelopeDifferential`	differential
4	Strkey	`FuzzStrkeyDifferential`	differential
6	`pkg/iso20022` XML	`FuzzParseDocuments`, `FuzzMarshalUnmarshalRoundTrip`	local
7	`pkg/x402` headers/amounts	`FuzzDecodeHeaders`, `FuzzParseAtomicAmount`	local
8	`internal/mcp` dispatch	`FuzzCallTool`, `FuzzValidateAndCollect`	local
9	`internal/config` state	`FuzzConfigLoad`, `FuzzLoadState`	local

We also added a small Tier-1 regression corpus containing eight representative artifacts under `pkg/soroban/testdata/diff-corpus/`. It covers ScVal scalars, 128-bit boundary values, nested maps and vectors, contract errors, account and contract addresses, valid and mutated strkeys, and a testnet transaction envelope.

All eight artifacts are replayed against both implementations during the test run. The corpus also includes a malformed strkey to check that invalid input is handled consistently.

4 Results

4.1 Differential Fuzzing

These tests use the Rust CLI as an external reference.

Target	Executions	Outcome
`FuzzScValEncodeDifferential`	449 (30 s)	No divergence
`FuzzScValDecodeDifferential`	∼36 k (25 s)	1 divergence, triaged (Section 5)
`FuzzStrkeyDifferential`	50 (20 s)	No divergence
`FuzzTxEnvelopeDifferential`	151 (15 s)	No divergence
Corpus replay	8 artifacts	Byte-equal and JSON-equal in both directions

4.2 Local Fuzzing

These tests run directly against the Go implementation, without an external reference process.

Target	Executions	Outcome
`FuzzCompareScVal`	344 k (15 s)	No ordering-law violations
`FuzzCallTool`	536 k (20 s)	No panics or schema bypasses
`FuzzParseDocuments`	274 k (20 s)	No panics
`FuzzDecodeHeaders`	465 k (15 s)	No panics
`FuzzParseAtomicAmount`	628 k (15 s)	No panics or negative amounts
`FuzzLoadState`	21 k (10 s)	No panics

The end-to-end test setup also passed:

- `make fuzz FUZZTIME=5s` – all 14 fuzz targets passed.
- `make diff-corpus` – all eight regression artifacts passed.
- `TestPlantedDivergence` – confirmed that the comparison logic detects deliberately corrupted oracle output.

The last test is particularly useful: it confirms that the harness does not simply report success when the two implementations disagree.

5 Findings

5.1 Divergence D1: Different XDR Decoding Strictness

Status: Triaged. This is an SDK behaviour difference, not a confirmed bug in the Go CLI.

The fuzzer found one input that the Go SDK accepts but the Rust reference rejects:

```
00 00 00 00 00 00 00 00 30
```

The two implementations behave differently:

Implementation	Behaviour	Explanation
Go SDK `UnmarshalBinary`	Accepts; decodes `ScvBool(false)`	Reads the union head but ignores trailing byte `0x30`
Rust `stellar xdr decode`	Rejects: "xdr value invalid"	Requires the entire input buffer to be consumed

We traced this difference to the Go XDR decoder, which allows trailing bytes, whereas the Rust decoder requires an exact buffer match.

This is a difference in decoder strictness rather than a difference in how the two implementations interpret a valid ScVal.

To avoid reporting these cases as semantic divergences, the differential harness now checks whether a decoded input can be re-encoded into exactly the same bytes. Inputs that fail this round-trip check are logged and excluded from semantic comparison.

This boundary is also documented in the test design.

5.2 Harness Defect H1: Incorrect ScError Union Arm

The regression corpus also helped catch a bug in our own test-only serde codec.

For contract errors, the codec was populating the generic `Code *ScErrorCode` field instead of the `ContractCode *Uint32` field of `xdr.ScError`.

This was a problem in the test harness, not in the production implementation. It was corrected and the resulting encoding was verified byte-for-byte against the Rust reference.

Catching this early was important because an incorrect test codec could otherwise produce misleading differential results.

5.3 What the Tests Did Not Find

Within the tested input space, we found no remaining semantic divergences between the two implementations.

More specifically:

- **ScVal encoding:** XDR output was byte-identical for all generated variants, including bool, void, u32/i32, u64/i64, timepoint, duration, u128/i128, bytes, string, symbol, vec, map, address and contract error.
- **Strkeys:** Acceptance/rejection behaviour and decoded payload bytes matched, including the mutated-checksum case.

- **Transaction hashing:** `tx.HashHex` and `stellar tx hash` produced matching results for all generated payment envelopes using the testnet passphrase.
- **Transaction decoding:** The Rust CLI successfully decoded every envelope produced by the Go implementation.

These results apply to the variants and cases covered by the current campaigns; they should not be interpreted as proof of equivalence for every possible input.

5.4 Additional Properties Verified Locally

Some parts of the implementation do not have a suitable Rust comparison oracle. For these, we tested expected behaviour directly.

- **ScVal ordering:** `CompareScVal` satisfied reflexivity, antisymmetry, transitivity (checked using triples) and totality across 344k executions. This matters because `stellar xdr encode` preserves the input order of ScMap entries and therefore cannot be used as an ordering oracle.
- **Canonicalization:** `ScvMap` produced identical XDR bytes regardless of whether map entries were supplied in forward or reversed order.
- **ISO 20022 round trips:** Parsing, marshaling, parsing again and marshaling once more produced a stable result.
- **MCP dispatch:** The central `callTool` dispatch function rejected malformed parameters and unknown tools without forwarding them to a handler. It also consistently returned valid JSON-RPC responses containing exactly one of `result` or `error`.
- **General robustness:** No panics or hangs were observed across the local fuzz targets.

6 Limitations and Areas for Further Testing

The results are encouraging, but there are still areas that need broader coverage.

1. **Campaign duration.** Differential tests are currently short smoke tests. Starting the Rust CLI for every input limits throughput to roughly 10–60 executions per second, compared with around 25k executions per second for local tests.

   As a result, "zero divergence" describes the tested campaigns, not an exhaustive verification. Longer runs are handled by `fuzz-nightly.yml` (5–10 minutes per target) and can also be run locally using `make fuzz FUZZTIME=30m`.

2. **Input coverage.** The test-only serde codec currently supports only the generated ScVal subset. Ledger entries, contract-spec entries, muxed accounts and claimable-balance strkeys have not yet been included in differential testing.

   Amount/decimal handling (target #5), VCs and swap-related functionality also remain outside the current campaigns.

3. **Ordering comparison.** We verify `CompareScVal` through ordering properties rather than a byte-level comparison with Rust map canonicalization. The Rust encoder preserves map input order, so it does not provide an independent ordering reference.

4. **Phase 4 still outstanding.** We have not yet compared simulation and invocation outcomes against a local network using `stellar container`.

5. **Trailing-byte handling.** D1 is a genuine difference in decoder behaviour. If the implementation later accepts XDR data from untrusted external sources, it would be worth adding a defensive length check after decoding. This has been flagged for security review rather than patched here, since the current decoding path is internal/test-side.

7 Reproducing the Tests

The following commands reproduce the main checks:

```
make diff-corpus                          # Replay the Tier-1 corpus
cd pkg && go test -run TestCorpusDifferential -v ./soroban/
cd pkg && go test -fuzz=FuzzScValEncodeDifferential -fuzztime=5m ./soroban/
cd pkg && go test -fuzz=FuzzCompareScVal -fuzztime=5m ./soroban/
go test -fuzz=FuzzCallTool -fuzztime=5m ./internal/mcp/
make fuzz FUZZTIME=5s                     # Run all 14 targets
```